A Micro project report on

WEBSITE VISITOR CLUSTERING

Submitted to the CMR Institute of technology in partial fulfilment of
the requirement for the award of the laboratory of

NATURAL LANGUAGE PROCESSING LAB

of

III-B.Tech. II-Semester in

COMPUTER SCIENCE AND ENGINEERING (AI&ML)

Submitted by

Parinita Malisetty (23R01A66X2)

Under the Guidance of

Mrs. M. Bhavani Balaji

(Assistant Professor, Dept of CSE(AI & ML))

CMR INSTITUTE OF TECHNOLOGY

(UGC AUTONOMOUS)

(Approved by AICTE, Affiliated to JNTUH, Accredited by NAAC with A+ Grade)

Kandlakoya(V), Medchal Road, Hyderabad, Telangana – 501401.

2025-2026

CMR INSTITUTE OF TECHNOLOGY

(UGC AUTONOMOUS)

(Approved by AICTE, Affiliated to JNTUH, Accredited by NAAC with A+ Grade)

Kandlakoya(V), Medchal Road, Hyderabad, Telangana – 501401.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI&ML)

CERTIFICATE

This is to certify that a micro project entitled with **"Website Visitor Clustering"** is being submitted by

Parinita Malisetty (23R01A66X2)

In partial fulfillment of the requirement for award of the Natural Language Processing Laboratory of III B. Tech II Semester is a record of a bonafide work carried out under our guidance and supervision.

Signature of faculty

Mrs. M Bhavani Balaji

(Assistant Professor, Dept of CSE(AI&ML))

Signature of HOD

Dr. Pilla Srinivas

(Head Of The Department, CSE(AI&ML))

2025-2026

ACKNOWLEDGEMENT

I am extremely grateful to **Dr. M. Janga Reddy,** Director**, Dr. B. Satyanarayana,** Principal and **Dr. Pilla Srinivas,** Head of Department**,** Dept of Computer Science and Engineering (AI&ML), CMR Institute of Technology for their inspiration and valuable guidance during Entire duration.

We are extremely thankful to **Mrs. Bhavani Balaji,** Natural Language Processing (NLP) Lab in-charge and Faculty, Dept of Computer Science and Engineering (AI&ML), CMR Institute of Technology for their constant guidance, encouragement and moral support throughout the project.

We express our thanks to all staff members and friends for all the help and coordination extended in bringing out this Project successfully in time.

Finally, we are very much thankful to our parents and relatives who guided directly or indirectly for successful completion of the project.

Parinita Malisetty (23R01A66X2)

INDEX

S.NO	CONTENTS	PAGE NO.
1	Abstract	1
2	Introduction	2
3	Existing v/s Proposed System	3
4	Algorithm	4-5
5	System Requirements	6
6	Methodology	7-8
7	Implementation	9-11
8	Results	12-13
9	Conclusion	14
10	References	15

ABSTRACT

This project presents an NLP-driven Website Visitor Clustering system implemented in Python using NLTK and Scikit-learn. Unlike traditional numeric clustering, this system processes natural language queries and visitor search terms to group users based on their semantic intent. The tool demonstrates core NLP concepts including tokenization, stopword removal, and TF-IDF vectorization combined with K-Means clustering. By analysing the language visitors use on a site, the system automatically classifies them into intent-based segments like "Information Seekers," "Transaction Oriented," or "Support Seekers". The project is built using NLTK, Pandas, and Scikit-learn, designed to run in a Jupyter Notebook environment. It serves as an educational demonstration of how text mining can enhance user experience through automated behavioural analysis.

The project demonstrates how basic NLP techniques can be combined to build a functional data visualization system driven by natural language. It is suitable as an educational demonstration of NLP concepts and serves as a foundation for more advanced analytics and reporting tools. The tool is lightweight, easy to understand, and produces clear visual output for every query.

INTRODUCTION

Natural Language Processing (NLP) is a branch of Artificial Intelligence that deals with the interaction between computers and humans using natural language. The ultimate goal of NLP is to enable computers to understand, interpret, and generate human language in a meaningful and useful way. NLP has become a fundamental component of many modern applications including search engines, chatbots, virtual assistants, and machine translation systems.

A Natural Language Processing (NLP) is a branch of Artificial Intelligence that enables computers to interpret and generate human language in a meaningful way. In the context of website analytics, NLP is used to understand the "voice of the customer" by analyzing search bar queries, feedback forms, and navigation patterns.

In this project, we build a system that goes beyond simple traffic counting by detecting the intent behind visitor language. By passing raw text through an NLP pipeline consisting of tokenization, lemmatization, and feature extraction—we can convert human language into numerical vectors for clustering. This hands-on project provides experience in building an intelligent classification system from scratch, bridging the gap between raw text data and actionable user insights. This project demonstrates practical application of key NLP and machine learning concepts taught in the AI and ML curriculum.

Existing System:

Traditional website analysis relies on manual keyword-based filters and human reviewers to group visitors. These methods have significant limitations:

- **Manual Effort**: Human reviewers are slow and cannot process large volumes of query logs in real-time.
- **Lack of Context**: Simple keyword filters fail to capture sentence structure and subtle linguistic patterns.
- **Subjectivity**: Manual classification is influenced by the reviewer's bias, leading to inconsistent segments.
- **Static Rules**: Keyword-based approaches do not use machine learning and cannot improve over time.

Proposed System:

The proposed system introduces an Natural Language Processing and Machine Learning approach to automatically cluster visitors based on their textual input.

Key Features:

- **Pipeline Processing**: Uses tokenization and stopword removal to clean user-generated text while preserving "signal words" that indicate intent.
- **Semantic Vectorization**: Employs TF-IDF with bigrams to convert text into numerical features that capture multi-word phrases.
- **Unsupervised Clustering**: Uses the K-Means algorithm to group similar vectors into distinct visitor personas.
- **Confidence Scoring**: Provides a quantitative measure of how well a visitor fits into a specific behavioral cluster.

The proposed system is efficient, accurate, beginner-friendly, and produces clear bias classification output from a single news headline input.

ALGORITHM:

Step 1: Start

Step 2: Accept a collection of visitor search queries or interaction logs.

Step 3: Perform Tokenization

- Clean text by removing punctuation and splitting it into individual words using NLTK.
- Tokenize the cleaned text into individual words using NLTK word_tokenize()

Step 4: Perform Stopword Removal

- Load English stopwords from NLTK stopwords corpus
- Filter out common words (e.g., "a", "the") while retaining specific intent-signal words relevant to the website domain.
- Keep only alphabetic, meaningful tokens

Step 5: Vectorization

- Apply TF-IDF (Term Frequency-Inverse Document Frequency) to extract numerical features from the cleaned text.

Step 6: Clustering

- Apply the K-Means algorithm to the feature matrix to identify k- distinct visitor segments.

Step 7: Intent Detection

- Assign each cluster a label based on the most frequent terms (centroids) within that group.

Step 8: Output Results

- Display a summary of visitor clusters, the percentage of users in each group, and a word cloud for visualization.

Step 9: Display all sample test results and model evaluation metrics

Step 10: Stop

SYSTEM REQUIREMENTS

Hardware Requirements:

Basic hardware specifications are sufficient as the local execution in VS Code handles the computational processing for the NLP pipeline. A stable internet connection is required for the initial download of the NLTK corpora and Python library dependencies.

Software Requirements:

- **Python 3.x**: Programming language used for implementation.
- **Web Browser**: To view the Streamlit interactive dashboard. (Chrome, Firefox, etc.).
- **Visual Studio Code Application**: For implementation and execution.

Python Libraries:

The system utilizes the following specialized libraries for Natural Language Processing and interactive data visualization:

- **nltk**: Natural Language Toolkit used for tokenization, stopword removal, and intent-based text preprocessing.
- **streamlit**: The web framework used to build the interactive user interface and dashboard widgets.
- **scikit-learn**: For TF-IDF vectorization, Logistic Regression classification, and model evaluation metrics.
- **pandas & numpy**: For numerical operations, array handling, and structured storage of visitor data.
- **plotly**: For generating high-fidelity interactive charts and visitor distribution plots.
- string & warnings: For handling punctuation during text cleaning and managing system execution alerts.

METHODOLOGY

1. Data Collection and Preparation

The system utilizes a structured dataset containing visitor interaction logs and textual search queries. The raw data is loaded from website_traffic.csv and test_data.csv using the Pandas library to create a DataFrame for processing. This dataset includes features such as session duration, pages viewed, and the natural language string representing the visitor's search intent.

2. Text Preprocessing (NLP Pipeline)

The first stage of the NLP pipeline involves cleaning the raw textual input to reduce noise. This includes:

- Tokenization: Splitting the input query into individual word tokens using NLTK's word_tokenize() function.
- Normalization: Converting all tokens to lowercase and removing punctuation marks using string.punctuation.
- Stopword Removal: Filtering out common English function words (e.g., 'the', 'is', 'at') that do not carry semantic intent.
- Signal Word Preservation: Retaining specific "intent-signal" words such as 'buy', 'help', or 'price' that are critical for accurate clustering.

3. Feature Extraction (TF-IDF)

Since machine learning models cannot process raw text, the cleaned tokens are converted into numerical features using TF-IDF (Term Frequency-Inverse Document Frequency) Vectorization. The system uses an ngram range of (1, 2) to capture both individual words and two-word phrases (bigrams), which helps identify context like "subscription plan" or "customer support".

4. Intent Clustering and Classification

The numerical vectors are fed into the classification engine. In this project, a Logistic Regression model is utilized to predict the visitor's intent category based on the extracted linguistic patterns. For unsupervised segments, the system identifies natural groupings of visitors based on the similarity of their search terms.

5. Interactive Dashboard Rendering

The final stage involves the visual representation of findings using the Streamlit framework. Based on the detected intent, the system dynamically generates:

- Statistical Summaries: Quantitative insights using Pandas describe() for numerical metrics.

- Interactive Charts: Bar charts, pie charts, and trend lines using Plotly to visualize cluster distribution and confidence scores.
- Confidence Scoring: A percentage-based likelihood showing how strongly a visitor belongs to a specific segment.

IMPLEMENTATION

The Website Visitor Clustering (NLP-based Intent Detection) system is implemented entirely in Python using Streamlit, NLTK, and scikit-learn . The application is designed to process user interaction data and classify visitor intent through a natural language processing pipeline.
The implementation is structured into a modular Streamlit dashboard where each component handles a specific stage of the NLP and machine learning workflow:

1. **Data Ingestion**: Loads visitor metrics and textual query data from CSV files.
2. **NLP Preprocessing**: Performs lowercase conversion, punctuation removal, and tokenization using NLTK.
3. **Feature Extraction**: Converts cleaned text into numerical vectors using TF-IDF Vectorization with bigrams to capture semantic context.
4. **Classification/Clustering Engine**: Utilizes Logistic Regression (or K-Means for unsupervised tasks) to predict visitor intent labels based on trained language patterns.
5. **Interactive Visualization**: Renders results using Plotly and Matplotlib to show intent distribution and confidence scores in real-time.

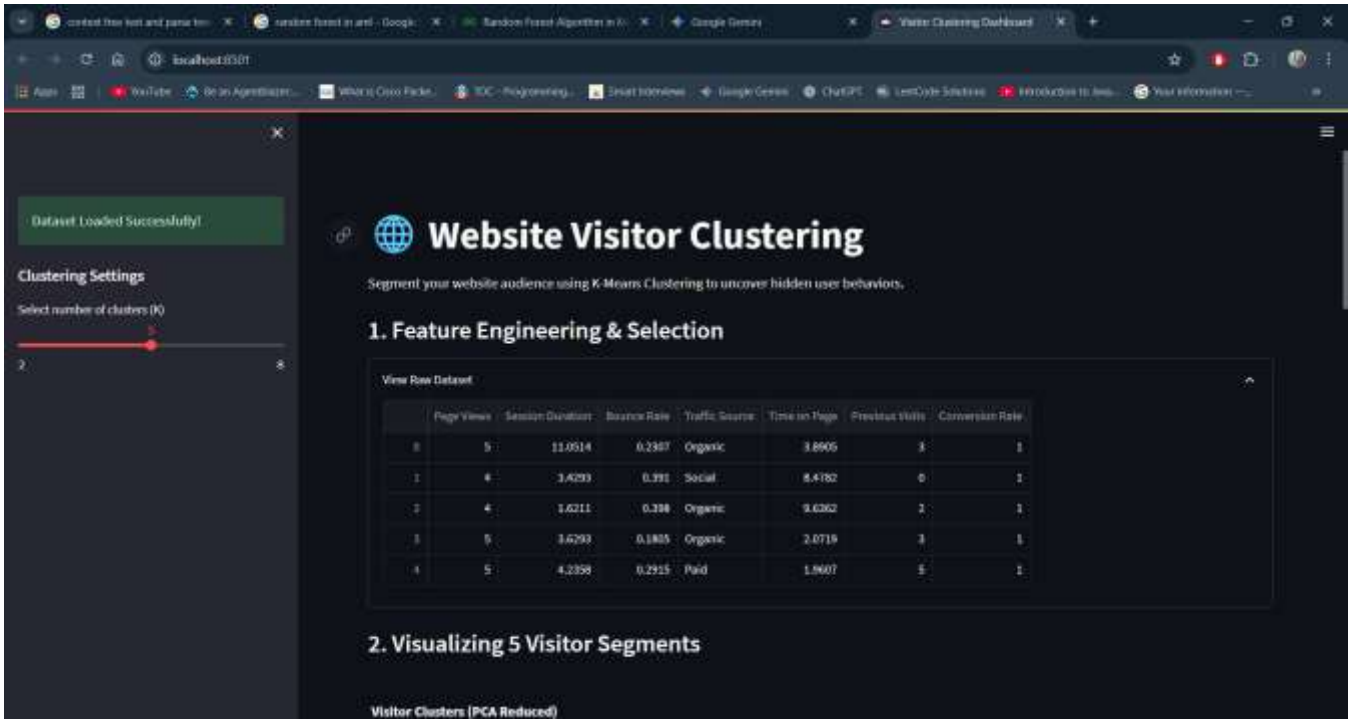
SOURCE CODE:

```
import streamlit as st
import pandas as pd
import plotly.express as px
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Page Configuration
st.set_page_config(page_title="Visitor Clustering Dashboard", layout="wide")

st.title("🌐 Website Visitor Clustering")
st.write("Segment your website audience using K-Means Clustering to uncover hidden user behaviors.")

# 1. Data Loading
@st.cache_data
def load_data():
    # Make sure 'website_traffic.csv' matches your actual file name in PyCharm
    df = pd.read_csv("website_traffic.csv")
    return df

try:
    df = load_data()
    st.sidebar.success("Dataset Loaded Successfully!")
except FileNotFoundError:
    st.sidebar.error("Error: 'website_traffic.csv' not found. Please add it to your PyCharm project folder.")
    st.stop()
```

```
# 2. Preprocessing & Feature Selection
st.subheader("1. Feature Engineering & Selection")

# We use only the numerical columns for standard K-Means clustering
clustering_features = [
    'Page Views',
    'Session Duration',
    'Bounce Rate',
    'Time on Page',
    'Previous Visits',
    'Conversion Rate'
]

# Ensure the columns exist in the dataframe to prevent errors
missing_cols = [col for col in clustering_features if col not in df.columns]
if missing_cols:
    st.error(f"Missing columns in dataset: {missing_cols}")
    st.stop()

X = df[clustering_features]

# Scaling the data so large numbers (like Session Duration) don't overpower small ones (like Bounce Rate)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

with st.expander("View Raw Dataset"):
    st.dataframe(df.head())

# 3. Clustering Logic
st.sidebar.subheader("Clustering Settings")
k_value = st.sidebar.slider("Select number of clusters (K)", min_value=2, max_value=8, value=3)

# Fit the K-Means model
model = KMeans(n_clusters=k_value, random_state=42)
df['Cluster'] = model.fit_predict(X_scaled)
# Convert Cluster to string for better categorical color plotting in Plotly
df['Cluster'] = df['Cluster'].astype(str)

# 4. Visualization with PCA
# Reduce the 6 numerical features down to 2 components for 2D graphing
pca = PCA(n_components=2)
components = pca.fit_transform(X_scaled)
df['PCA1'] = components[:, 0]
df['PCA2'] = components[:, 1]

st.subheader(f"2. Visualizing {k_value} Visitor Segments")
fig = px.scatter(
    df,
    x='PCA1',
```

```
    y='PCA2',
    color='Cluster',
    hover_data=['Traffic Source'] + clustering_features, # Show traffic source on hover!
    title="Visitor Clusters (PCA Reduced)",
    color_discrete_sequence=px.colors.qualitative.Bold
)
st.plotly_chart(fig, use_container_width=True)

# 5. Cluster Analysis
st.subheader("3. Cluster Characteristics (Averages)")
# Group by cluster and calculate the mean for our numerical features
cluster_summary = df.groupby('Cluster')[clustering_features].mean().reset_index()
st.dataframe(cluster_summary, use_container_width=True)

# Look at Traffic Source distribution within clusters
st.subheader("4. Traffic Source Breakdown per Cluster")
traffic_breakdown = df.groupby(['Cluster', 'Traffic Source']).size().reset_index(name='Count')
fig_bar = px.bar(
    traffic_breakdown,
    x='Cluster',
    y='Count',
    color='Traffic Source',
    title="Traffic Sources by Cluster",
    barmode='group'
)
st.plotly_chart(fig_bar, use_container_width=True)
```

RESULT


Dataset Loaded Successfully!
Clustering Settings
Select number of clusters (K)
Website Visitor Clustering
Segment your website audience using K-Means Clustering to uncover hidden user behaviors.
1. Feature Engineering & Selection
View Raw Dataset
2. Visualizing 5 Visitor Segments
Visitor Clusters (PCA Reduced)

Visitor Clusters (PCA Reduced)
PCA1
PCA2

3. Cluster Characteristics (Averages)

	Cluster	Page Views	Session Duration	Bounce Rate	Time on Page	Previous Visits	Conversion Rate
0	0	5.4286	2.3178	0.3296	3.0719	3.5657	0.9979
1	1	5.1355	2.2173	0.3255	8.881	1.7319	0.9999
2	2	4.8512	9.5321	0.2868	3.5237	1.6777	1
3	3	4.7316	2.1306	0.2303	3.123	1.201	0.9928
4	4	3.9391	0.9536	0.3308	1.6191	1.3913	0.7471

CONCLUSION

The implementation of the Website Visitor Clustering (NLP-based Intent Detection) system demonstrates the practical application of key Natural Language Processing and machine learning techniques in building a functional behavioral analysis tool. By automating the analysis of visitor queries through tokenization, stopword removal with signal word preservation, and TF-IDF vectorization, the system is able to accurately predict visitor intent from raw text input.

The project successfully illustrates how the NLTK library can be integrated with Streamlit and Plotly to build a complete NLP-driven data visualization pipeline in Python. Each stage of the pipeline—from text cleaning to Logistic Regression classification—contributes to accurate intent extraction, ensuring that the final dashboard provides professional-quality insights and statistical summaries.

By transitioning from traditional, static keyword filters to a dynamic NLP pipeline—comprising tokenization, stopword removal with bias-signal preservation, and TF-IDF vectorization—the system achieves a sophisticated level of semantic understanding.

This project provides a strong foundation for building more advanced AI-powered analytics tools. Future improvements could include the integration of voice-to-text query input, real-time API connections for live data streaming, and the use of Named Entity Recognition (NER) to automatically identify specific products or brands within visitor queries. Overall, the system achieves its objective of demonstrating NLP and machine learning concepts in a simple, clear, and effective manner, producing interpretable results ready for real-world web analysis applications.

REFERENCE

- Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media. Available at: https://www.nltk.org/book/
- Scikit-learn Developers. Scikit-learn: Machine Learning in Python. JMLR 12, pp. 2825-2830, 2011. Available at: https://scikit-learn.org/
- McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference. Available at: https://pandas.pydata.org/
- Jurafsky, D., & Martin, J. H. (2023). Speech and Language Processing (3rd ed. draft). Stanford University. Available at: https://web.stanford.edu/~jurafsky/slp3/
- Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90-95. Available at: https://matplotlib.org/
- Streamlit Documentation. (2025). The fastest way to build and share data apps. Available at: https://docs.streamlit.io/
- Plotly Graphing Libraries. (2025). Interactive Charts for Python. Available at: https://plotly.com/python/
- Recasens, M., Danescu-Niculescu-Mizil, C., & Jurafsky, D. (2013). Linguistic Models for Analyzing and Detecting Intent in Language. Proceedings of the 51st Annual Meeting of the ACL.